

cornea

Builds a 2D axisymmetric cornea section bounded by two conic surfaces, meshes it by mapping a structured square mesh onto it, exports a tagged Gmsh mesh, and recovers the Jacobian of the mapping to orient fields (fibres, normals) defined on the square.

Quick start

0. Compile README.MD

For readability, compile this readme in a pdf using pandoc :

```
pandoc README.md -o README.pdf
```

1. Create the Python environment

From the `cornea_meshing_package/` folder (the one containing `pyproject.toml`):

```
python3 -m venv cornea_msh_env          # create the environment
source cornea_msh_env/bin/activate      # activate it (Linux / macOS)
pip install --upgrade pip
pip install -e .                        # installs the package and its dependencies
```

On Windows, activate with `cornea_msh_env\Scripts\activate` instead of the source line.

`pip install -e .` reads the dependencies from `pyproject.toml` (numpy, scipy, matplotlib, meshio) and installs them inside `cornea_msh_env` only. The `-e` (editable) flag links the package to the source folder, so changes you make to the code apply without reinstalling.

Activate the environment again in each new terminal before using the package, and leave it with `deactivate`.

2. Run

```
python -m cornea params.json -o output
```

`params.json`:

```
{
  "R_v": 5.25,
  "R_a": 6.5,  "Q_a": -0.61,
  "R_p": 5.5,  "Q_p": -0.72,
  "h": 0.5,
  "Nx": 50,  "Ny": 6
}
```

Parameter	Meaning
R_v	radial extent of the posterior face
R_a, Q_a	anterior radius of curvature and asphericity
R_p, Q_p	posterior radius of curvature and asphericity
h	central thickness
Nx, Ny	number of nodes along the radius / through the thickness

Outputs in output/:

File	Content
<code>cornea.msh</code>	Gmsh MSH 2.2 ASCII mesh: nodes, 4-node quads, tagged boundary lines, nodal fields u, v
<code>cornea_mesh.png</code>	the mapped mesh
<code>cornea_fields.png</code>	fibre and normal directions, computed from <code>cornea.msh</code> only
<code>cornea_tags.png</code>	physical groups, read back from <code>cornea.msh</code>

The console also prints checks: the limbus edge is normal to the posterior face, the discrete Jacobian matches the exact one, and every element keeps a positive orientation.

Physical groups in `cornea.msh`

Name	Dimension	Tag
<code>anterior</code>	1	1
<code>posterior</code>	1	2
<code>limbus</code>	1	3
<code>central_line</code>	1	4
<code>cornea</code>	2	10

Coordinates are written as (x, z, 0): x is the radial direction, z the optical axis.

Using it from Python

```
from cornea import CorneaParams, CorneaGeometry, build_nodes, write_msh
```

```

p = CorneaParams.from_json("params.json")      # or CorneaParams(R_v=5.0, ...)
geom = CorneaGeometry(p)
pts, uv = build_nodes(geom, p.Nx, p.Ny)        # (Ny, Nx, 2) arrays, pts = phi(uv)
write_msh("cornea.msh", pts, uv)

xz = geom.phi(0.5, 0.5)                        # exact map at any (u, v)
F = geom.F_exact(0.5, 0.5)                     # its Jacobian

```

1. Geometry: the two conics

Each face is a conic sag of radius R and asphericity Q :

$$f(x; R, Q) = \frac{x^2/R}{1 + \sqrt{1 - (1 + Q)x^2/R^2}}, \quad f'(x; R, Q) = \frac{x}{\sqrt{R^2 - (1 + Q)x^2}}$$

The faces are placed directly in their final position (apex up, opening downwards):

$$z_{ant}(x) = T - f(x; R_a, Q_a), \quad z_{post}(x) = T - h - f(x; R_p, Q_p), \quad T = h + f(R_v; R_p, Q_p)$$

so the central thickness is h and the posterior end point $P = (R_v, 0)$ sits at $z = 0$.

Limbus edge. Instead of cutting vertically at $x = R_v$, the edge follows the normal to the posterior face at P . With $s = z'_{post}(R_v)$, the unit normal pointing towards the anterior face is

$$n = \frac{(-s, 1)}{\sqrt{1 + s^2}}$$

and the anterior end point is $A = P + t^*n$, where t^* solves $z_{ant}(P_x + t n_x) = P_z + t n_z$ (Brent's method). The anterior face therefore extends slightly beyond R_v (to $x \approx 5.73$ with the default parameters).

2. Mapping: square to cornea (Coons patch)

The reference square has coordinates $(u, v) \in [0, 1]^2$:

- u : radial, $u = 0$ on the central line, $u = 1$ on the limbus;
- v : through the thickness, $v = 0$ on the posterior face, $v = 1$ on the anterior face.

The four boundaries of the cornea are parametrised on $[0, 1]$:

- $C_{post}(u)$, $C_{ant}(u)$: the two conics, parametrised by normalised arc length (cubic spline of $x(s)$), so nodes are evenly spaced along each face;

- $D_{axis}(v)$, $D_{limb}(v)$: straight segments between the corner points $P_{00} = C_{post}(0)$, $P_{01} = C_{ant}(0)$ and $P_{10} = C_{post}(1) = P$, $P_{11} = C_{ant}(1) = A$.

The map is the transfinite (Coons) interpolation of these four curves:

$$\varphi(u, v) = (1-v) C_{post}(u) + v C_{ant}(u) + (1-u) D_{axis}(v) + u D_{limb}(v) - [(1-u)(1-v)P_{00} + u(1-v)P_{10} + (1-u)v P_{01} + uv P_{11}]$$

It reproduces each boundary exactly and blends them smoothly inside. Node (i, j) of an $N_x \times N_y$ grid on the square is placed at $(x, z) = \varphi(i/(N_x - 1), j/(N_y - 1))$.

Because u points outward and v points from posterior to anterior (upward), φ preserves orientation ($\det F > 0$): quads that are counter-clockwise in (u, v) are counter-clockwise in (x, z) . The writer checks this and raises an error on any inverted element.

If your solver works with displacements instead, `geom.displacement(X, Y, L, H)` gives $\varphi(X/L, Y/H) - (X, Y)$ for nodes of a square $[0, L] \times [0, H]$; keep only the boundary nodes to use it as a Dirichlet condition.

3. The Jacobian F and how to use it

Definition

$$F = \frac{\partial(x, z)}{\partial(u, v)} = \begin{bmatrix} \partial x / \partial u & \partial x / \partial v \\ \partial z / \partial u & \partial z / \partial v \end{bmatrix}$$

Recovering F from the mesh file

Every node stores its physical coordinates (x, z) and, as nodal fields, its reference coordinates (u, v) , with $(x, z) = \varphi(u, v)$. No analytic φ is needed: inside each element both are interpolated with the same bilinear shape functions $N_a(\xi, \eta)$, so

$$J_x = \sum_a (x_a, z_a) \otimes \nabla_\xi N_a, \quad J_u = \sum_a (u_a, v_a) \otimes \nabla_\xi N_a, \quad F = J_x J_u^{-1}$$

This is `element_jacobians()` (evaluated at element centres). It works for any mesh that carries the `u`, `v` fields. The run compares it with the exact F of φ (relative error about 2e-5 with the default mesh).

Inside an FE solver it is even simpler: load `u` and `v` as FE functions and take their gradients. $G = [\nabla u; \nabla v] = F^{-1}$, so $F = G^{-1}$ and $F^{-T} = G^T$.

Transformation rules

Quantity defined on the square	On the cornea	Function
scalar $s(u, v)$ (grading, damage...)	$s(u(x), v(x))$: just evaluate at the node's (u, v)	
material direction A (fibre)	$a = FA / \ FA\ $	<code>push_vector(F, A)</code>
normal or gradient direction N	$n = F^{-T}N / \ F^{-T}N\ $	<code>push_normal(F, N)</code>
gradient of a scalar	$\nabla_x s = F^{-T} \nabla_{uv} s$	

Examples, as plotted in `cornea_fields.png`:

- **fibre along the radius:** $F(1, 0)$, tangent to the $v = \text{const}$ curves (including both faces);
- **normal from posterior to anterior:** $F^{-T}(0, 1) \propto \nabla v$, exactly normal to the anterior and posterior faces.

These two are orthogonal everywhere by construction ($F^T F^{-T} = I$), a useful check.

Pitfall: the mapped v -grid lines $F(0, 1)$ are *not* normal to the faces (up to about 3° off here), because the Coons map is not orthogonal. Use F^{-T} for anything that must be normal to the surfaces.

Axisymmetric note: the circumferential direction e_θ is not affected by the 2D map; for a fibre with radial and hoop components, map only the in-plane part with F .

Package layout

```
cornea/
  params.py      CorneaParams: inputs, JSON read/write
  geometry.py    conics, limbus edge, Coons map phi, exact F
  mesh_io.py     structured nodes, Gmsh MSH 2.2 writer (tags + u, v fields)
  jacobian.py    F from a meshio mesh, push_vector / push_normal, checks
  plots.py       plot_mesh, plot_fields, plot_tags
  __main__.py    command line: python -m cornea params.json -o output
params.json
pyproject.toml
```